

Informe final de DSL y herramientas

JUnit4, Mockito, ArchUnit y Vert.x

Patrones de Software y Programación

Propósito del informe. Consolidar en un solo documento los lenguajes de dominio específico (DSL) y APIs más relevantes para resolver ejercicios de arquitectura, testing automatizado y programación reactiva: reglas ArchUnit, tests JUnit4, mocks con Mockito y comunicación reactiva con Vert.x.

Enfoque. El documento no es un resumen superficial: funciona como formulario ampliado, guía de implementación y manual de corrección de errores frecuentes. Incluye patrones de lectura, tablas de métodos imprescindibles, ejemplos completos y criterios de diseño para ejercicios de evaluación.

Autoría: material elaborado para NicoBeli

Fecha: 23 de junio de 2026

Formato: LaTeX compilado a PDF

Índice

1. Resumen ejecutivo	4
1.1. Mapa mental de decisión rápida	4
2. Fundamentos arquitecturales que sostienen los DSL	6
2.1. Arquitectura como restricción de dependencias	6
2.2. Patrones arquitecturales mínimos	6
2.3. Relación entre arquitectura, tests y DSL	7
3. JUnit4: métodos y anotaciones que no pueden faltar	8
3.1. Modelo mental	8
3.2. Anotaciones imprescindibles	8
3.3. Aserciones esenciales	9
3.4. Testing de excepciones en JUnit4	10
3.4.1. Forma simple: @Test(expected = ...)	10
3.4.2. Forma más rigurosa: ExpectedException	10
3.5. Tests parametrizados	11
3.6. Principios FIRST aplicados	11
3.7. Checklist JUnit4 antes de entregar	12
4. Mockito: métodos imprescindibles y patrones de uso	13
4.1. Modelo mental	13
4.2. Métodos que no pueden faltar	13
4.3. Ejemplo base: servicio con repositorio	14
4.4. Test con Mockito JUnit4 Runner	15
4.5. Versión rigurosa para excepción y never()	16
4.6. Matchers: regla de consistencia	16
4.7. Métodos void: doThrow, doNothing, doAnswer	16
4.8. ArgumentCaptor: cuándo usarlo	17
4.9. Spy: uso mínimo y advertencia	17
5. ArchUnit: DSL arquitectural imprescindible	19
5.1. Qué resuelve ArchUnit	19
5.2. Dependencias Maven	19
5.3. Plantilla JUnit4 de ArchUnit	19
5.4. Por qué usar ImportOption.DoNotIncludeTests.class	20
5.5. Patrón S-R-C-E-X para escribir reglas	20
5.6. Patrones de paquetes	20
5.7. DSL base: classes(), noClasses(), that(), should()	21
5.8. Reglas de naming	21
5.9. Arquitectura por capas clásica	22
5.10 Reglas de acceso directo	22
5.11 Dominio puro	22

5.12Slices: detección de ciclos	22
5.13Arquitectura hexagonal / onion	23
5.14Reglas hexagonales explícitas	23
5.15Condiciones personalizadas	23
5.16Freezing rules	24
5.17Checklist ArchUnit	24
6. Vert.x: programación reactiva y EventBus	25
6.1. Modelo mental	25
6.2. Métodos esenciales de Vert.x	25
6.3. Point-to-Point con send	26
6.4. Publish/Subscribe con publish	27
6.5. Request-Response con request y reply	28
6.6. Future y Promise: composición asíncrona	29
6.7. Reglas de diseño Vert.x	30
6.8. Test de Vert.x con JUnit4 y vertx-unit	30
7. Integración: ejemplo compacto de arquitectura + JUnit + Mockito + ArchUnit + Vert.x	32
7.1. Estructura de paquetes recomendada	32
7.2. Regla de dependencia esperada	32
7.3. Servicio de aplicación	32
7.4. Test unitario con Mockito	33
7.5. Adaptador Vert.x de salida	34
7.6. Test del adaptador con Mockito	34
7.7. Test arquitectural	35
8. Tablas tipo formulario	36
8.1. JUnit4 en una página	36
8.2. Mockito en una página	36
8.3. ArchUnit en una página	37
8.4. Vert.x EventBus en una página	37
9. Errores frecuentes y cómo corregirlos	39
9.1. Errores JUnit4	39
9.2. Errores Mockito	39
9.3. Errores ArchUnit	39
9.4. Errores Vert.x	40
10Estrategia de resolución para pruebas del curso	41
10.1Lectura del enunciado	41
10.2Orden recomendado de implementación	41
10.3Cómo saber si un test es unitario	41
10.4Cómo saber si una regla ArchUnit está bien planteada	42
10.5Cómo decidir entre send, publish y request	42

11Apéndice A: dependencias Maven de referencia	43
12Apéndice B: plantilla completa de test arquitectural	44
13Apéndice C: plantilla completa Mockito	44
14Conclusión	46
Referencias	47

1 Resumen ejecutivo

Idea clave

La idea transversal de la unidad es que una arquitectura no se protege solo con diagramas. Se protege cuando sus decisiones se vuelven ejecutables: tests unitarios para comportamiento, mocks para aislar dependencias, reglas ArchUnit para dependencias estructurales y Vert.x para expresar comunicación reactiva entre componentes.

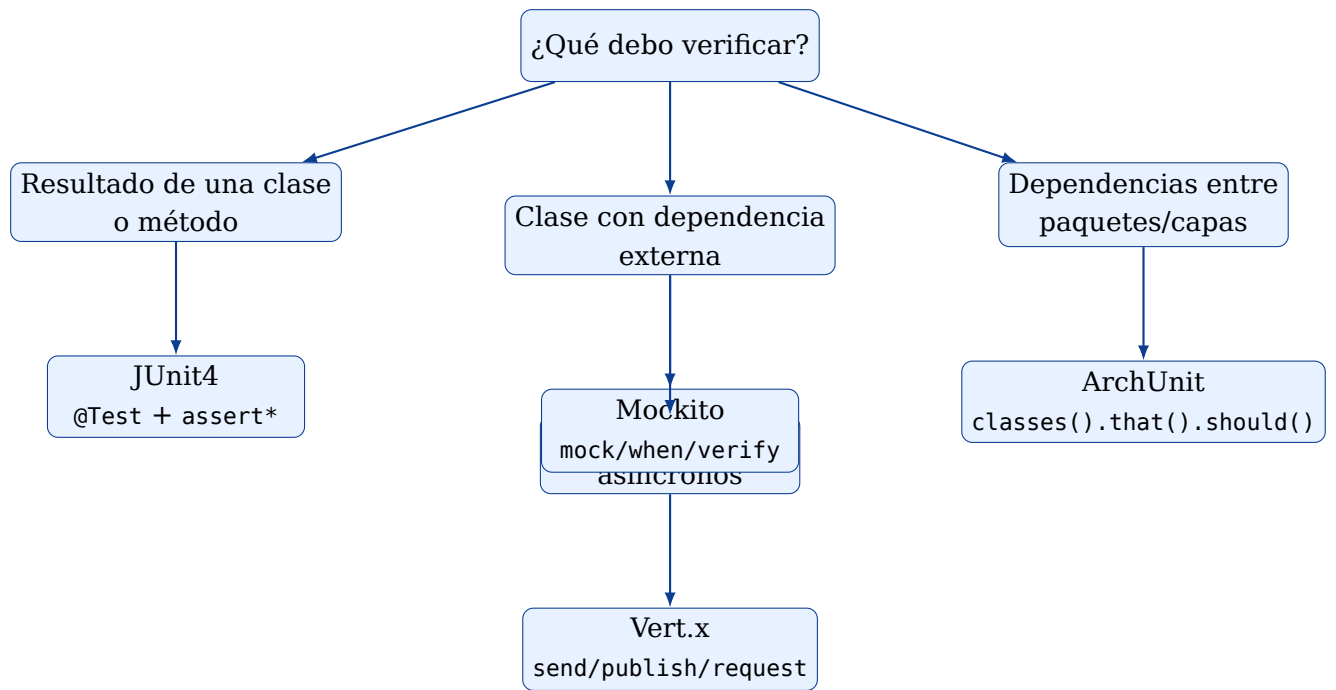
Este informe recopila los DSL y métodos más usados en el segundo bloque del curso. La palabra *DSL* se usa aquí en sentido práctico: una sintaxis fluida que permite expresar una intención técnica de alto nivel. En estos contenidos aparecen cuatro familias:

1. **JUnit4**: DSL mínimo de prueba, basado en anotaciones y aserciones. Su objetivo es declarar qué método es un test, preparar el fixture y verificar el resultado esperado.
2. **Mockito**: DSL de dobles de prueba. Permite crear objetos simulados, programar comportamiento, verificar interacciones y capturar argumentos.
3. **ArchUnit**: DSL de arquitectura ejecutable. Permite declarar reglas sobre paquetes, capas, dependencias, ciclos, nombres, arquitectura onion/hexagonal y excepciones controladas.
4. **Vert.x**: API reactiva basada en verticles, event loop y EventBus. Aunque no siempre se le llame DSL, su EventBus define un vocabulario expresivo: send, publish, request, consumer, reply.

En examen

Si la prueba pide “implemente la solución”, normalmente no basta con que el código compile. Debes demostrar tres cosas: **comportamiento correcto** con JUnit, **aislamiento** con Mockito cuando hay dependencias externas, y **respeto arquitectural** con ArchUnit. Si aparece Vert.x, además debes escoger correctamente el tipo de mensajería: send para punto a punto, publish para difusión y request/reply para petición-respuesta.

1.1 Mapa mental de decisión rápida



2 Fundamentos arquitecturales que sostienen los DSL

2.1 Arquitectura como restricción de dependencias

Una arquitectura no es un dibujo decorativo. Es un conjunto de decisiones sobre módulos, responsabilidades y dependencias permitidas. Las reglas del curso se entienden mejor si se separan dos conceptos:

- **Flujo de control:** quién llama a quién durante la ejecución.
- **Flujo de dependencias:** qué clase o paquete conoce, importa o depende de qué otra clase o paquete.

En una arquitectura por capas simple, ambos flujos suelen coincidir: Controller ->Service ->Repository. En una arquitectura hexagonal, pueden no coincidir: el caso de uso llama a un puerto de salida, pero el adaptador de persistencia implementa ese puerto. Por eso el adaptador conoce al núcleo, pero el núcleo no conoce al adaptador.

Idea clave

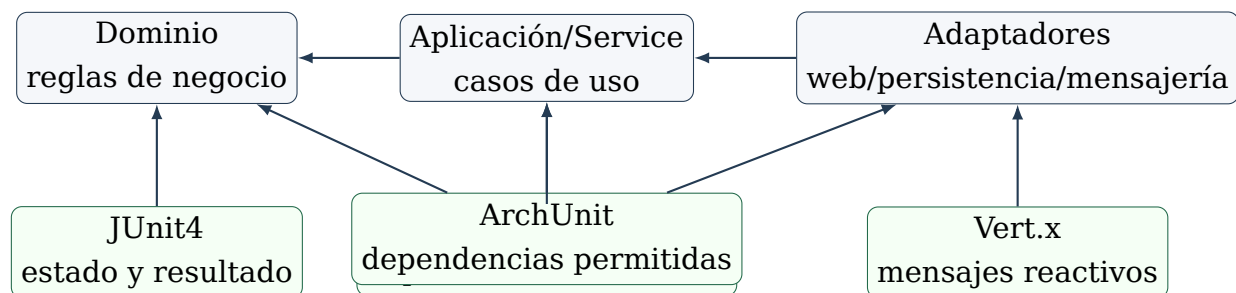
ArchUnit evalúa principalmente el flujo de dependencias, no el flujo dinámico de ejecución. Por tanto, una regla puede fallar aunque “funcione” en runtime, porque lo que está mal es la dirección del acoplamiento.

2.2 Patrones arquitecturales mínimos

Patrón	Idea central	Riesgo típico en ejercicios
Dos capas	Cliente combina interfaz y lógica; servidor suele concentrar datos.	Poner reglas de negocio en la interfaz o acoplar demasiado al almacenamiento.
Tres capas	Presentación, aplicación/negocio y datos.	Saltarse la capa intermedia: controlador llamando directamente a repositorio.
MVC tradicional	Modelo, vista y controlador para interfaz gráfica.	Confundir MVC desktop con MVC web; en web suele parecerse a tres capas.
Microservicios	Módulos ejecutan como procesos independientes.	Compartir base de datos entre servicios o convertir todo en microservicios sin necesidad.
Orientada a mensajes	Componentes se comunican mediante broker/EventBus.	Usar request-response cuando conviene evento, o bloquear esperando resultados.

Patrón	Idea central	Riesgo típico en ejercicios
Publish/Subscribe	Un productor publica un evento y muchos suscriptores lo reciben.	Esperar respuesta directa del publicador; Pub/Sub no es RPC.
Hexagonal	El dominio queda protegido; adaptadores externos se conectan por puertos.	Que el dominio dependa de JPA, Spring, HTTP, DTOs externos o repositorios concretos.

2.3 Relación entre arquitectura, tests y DSL



3 JUnit4: métodos y anotaciones que no pueden faltar

3.1 Modelo mental

JUnit4 ejecuta métodos marcados con `@Test`. Cada test debe tener una intención clara: preparar datos, ejecutar una unidad y verificar una salida observable. La estructura recomendada es *Arrange - Act - Assert*.

Receta

Plantilla mínima de test unitario JUnit4

```
1 import org.junit.Before;
2 import org.junit.Test;
3 import static org.junit.Assert.*;
4
5 public class CalculatorTest {
6
7     private Calculator calculator;
8
9     @Before
10    public void setUp() {
11        calculator = new Calculator();
12    }
13
14    @Test
15    public void add_returnsSum_whenInputsArePositive() {
16        // Arrange
17        int a = 2;
18        int b = 3;
19
20        // Act
21        int result = calculator.add(a, b);
22
23        // Assert
24        assertEquals(5, result);
25    }
26 }
```

3.2 Anotaciones imprescindibles

Anotación	Uso	Cuándo aparece en ejercicios
@Test	Marca un método público como caso de prueba.	Siempre. Es el núcleo de JUnit4.

Anotación	Uso	Cuándo aparece en ejercicios
@Before	Ejecuta preparación antes de cada @Test.	Crear fixture limpio: objetos, mocks, listas, repositorios fake.
@After	Limpieza después de cada test.	Cerrar recursos, limpiar archivos temporales, resetear estado externo.
@BeforeClass	Preparación una vez antes de todos los tests. Debe ser static.	Recursos costosos compartidos. Usarlo poco en unit tests.
@AfterClass	Limpieza una vez al final. Debe ser static.	Cerrar servidor, base embebida o recurso global.
@Ignore	Deshabilita temporalmente un test.	Solo si la evaluación lo permite; normalmente no conviene.
@RunWith	Cambia el runner de JUnit.	Mockito, tests parametrizados, ArchUnit JUnit4, Vert.x Unit.
@Rule	Agrega comportamiento al test.	ExpectedException, TemporaryFolder, Timeout.

Error típico

No uses @BeforeClass para inicializar objetos que luego se modifican en tests. Eso rompe la independencia: un test puede dejar estado sucio para el siguiente. En unit testing, la regla general es fixture nuevo por test.

3.3 Aserciones esenciales

Método	Qué verifica	Ejemplo mínimo
assertEquals(e,a)	Igualdad por equals o por valor primitivo.	assertEquals(3, size);
assertNotEquals(e,a)	Que dos valores no sean iguales.	assertNotEquals(0, id);
assertTrue(cond)	Condición booleana verdadera.	assertTrue(list.isEmpty());
assertFalse(cond)	Condición booleana falsa.	assertFalse(user.isActive());

Método	Qué verifica	Ejemplo mínimo
<code>assertNull(obj)</code>	Referencia nula.	<code>assertNull(found);</code>
<code>assertNotNull(obj)</code>	Referencia no nula.	<code>assertNotNull(saved);</code>
<code>assertSame(e,a)</code>	Misma referencia de objeto.	<code>assertSame(cache, result);</code>
<code>assertNotSame(e,a)</code>	Referencias distintas.	<code>assertNotSame(dto, entity);</code>
<code>assertArrayEquals(e,a)</code>	Igualdad elemento a elemento en arreglos.	<code>assertArrayEquals(exp, act);</code>
<code>fail(msg)</code>	Fuerza fallo explícito.	<code>fail("Debe lanzar excepcion");</code>

En examen

En JUnit4 el orden conceptual de `assertEquals` es **esperado, actual**. Es decir: `assertEquals(valorEsperado, valorObtenido)`. Si lo inviertes, el test puede fallar igual, pero el mensaje de error queda menos claro.

3.4 Testing de excepciones en JUnit4

3.4.1 Forma simple: `@Test(expected = ...)`

```

1 @Test(expected = IllegalArgumentException.class)
2 public void createAccount_throwsException_whenBalanceIsNegative() {
3     new Account("A-1", -1000);
4 }
```

Ventaja: breve. Desventaja: no permite verificar el mensaje ni el punto exacto donde se lanza la excepción. Además, si varias líneas pueden lanzar la misma excepción, el test queda ambiguo.

3.4.2 Forma más rigurosa: `ExpectedException`

```

1 import org.junit.Rule;
2 import org.junit.Test;
3 import org.junit.rules.ExpectedException;
4
5 public class AccountTest {
6
7     @Rule
8     public ExpectedException thrown = ExpectedException.none();
9
10    @Test
11    public void createAccount_throwsClearMessage_whenBalanceIsNegative() {
```

```
12     thrown.expect(IllegalArgumentException.class);
13     thrown.expectMessage("balance must be non-negative");
14
15     new Account("A-1", -1000);
16 }
17 }
```

3.5 Tests parametrizados

Útiles cuando la misma regla se evalúa con varios casos. En JUnit4 requieren `@RunWith(Parameterized.class)`

```
1  import org.junit.Test;
2  import org.junit.runner.RunWith;
3  import org.junit.runners.Parameterized;
4
5  import java.util.Arrays;
6  import java.util.Collection;
7
8  import static org.junit.Assert.assertEquals;
9
10 @RunWith(Parameterized.class)
11 public class DiscountPolicyTest {
12
13     @Parameterized.Parameters(name = "amount={0}, expected={1}")
14     public static Collection<Object[]> data() {
15         return Arrays.asList(new Object[][] {
16             {0, 0},
17             {10000, 0},
18             {50000, 5},
19             {100000, 10}
20         });
21     }
22
23     private final int amount;
24     private final int expectedDiscount;
25
26     public DiscountPolicyTest(int amount, int expectedDiscount) {
27         this.amount = amount;
28         this.expectedDiscount = expectedDiscount;
29     }
30
31     @Test
32     public void discount_dependsOnAmount() {
33         DiscountPolicy policy = new DiscountPolicy();
34         assertEquals(expectedDiscount, policy.discountFor(amount));
35     }
36 }
```

3.6 Principios FIRST aplicados

Principio	Significado	Cómo cumplirlo
Fast	Tests rápidos.	Evitar red, base real, sleeps; usar mocks/fakes.
Independent	No dependen del orden.	No compartir estado mutable entre tests.
Repeatable	Determinísticos.	Controlar tiempo, aleatoriedad, concurrencia y datos externos.
Self-checking	Resultado automático verde/rojo.	Usar asserts; no depender de mirar consola.
Timely	Escritos temprano.	Idealmente antes o junto con la implementación.

Error típico

Un `System.out.println` no prueba nada. Puede ayudar a depurar, pero el test solo es auto-verificable si tiene aserciones o verificaciones automáticas.

3.7 Checklist JUnit4 antes de entregar

- Cada test tiene un nombre que expresa condición y resultado.
- Cada test tiene al menos una aserción o una verificación Mockito significativa.
- No hay dependencia del orden de ejecución.
- No se prueba lógica de negocio a través de infraestructura si puede probarse directamente.
- Los tests de excepción verifican el caso negativo relevante.
- La cobertura no se persigue como fetiche: se cubren ramas relevantes y casos límite.

4 Mockito: métodos imprescindibles y patrones de uso

4.1 Modelo mental

Mockito sirve para aislar la unidad bajo prueba. En vez de llamar a una base de datos, API remota o repositorio real, se usa un objeto simulado que responde de forma controlada. Así el test mide la lógica de la clase bajo prueba, no la disponibilidad de infraestructura.

Idea clave

Un mock no es solo un objeto falso. Es un colaborador controlado: se le puede programar comportamiento y luego verificar cómo fue usado.

4.2 Métodos que no pueden faltar

Método/Anotación	Función	Ejemplo
<code>mock(Clase.class)</code>	Crear mock manualmente.	<code>Repository repo = mock(Repository.class);</code>
<code>@Mock</code>	Crear mock por anotación.	<code>@Mock Repository repo;</code>
<code>@InjectMocks</code>	Injectar mocks en la clase bajo prueba.	<code>@InjectMocks Service service;</code>
<code>@RunWith(MockitoJUnitRunner.class)</code>	Inicializa anotaciones Mockito en JUnit4.	En la clase de test.
<code>when(...).thenReturn(...)</code>	Programar retorno.	<code>when(repo.find(id)).thenReturn(book);</code>
<code>when(...).thenThrow(...)</code>	Programar excepción.	<code>when(repo.find(id)).thenThrow(...);</code>
<code>doThrow(...).when(mock).void()</code>	Programar excepción en método void.	<code>doThrow(ex).when(repo).delete(id);</code>
<code>doReturn(...).when(spy)...</code>	Stubbing seguro en spies.	Evita llamar método real durante stubbing.
<code>verify(mock).method(...)</code>	Verificar interacción.	<code>verify(repo).save(book);</code>
<code>verify(mock, times(n))</code>	Verificar cantidad exacta.	<code>verify(bus, times(2)).send(...);</code>
<code>never()</code>	Verificar que no ocurrió.	<code>verify(repo, never()).save(any());</code>
<code>verifyNoMoreInteractions(...)</code>	No hubo llamadas adicionales.	Útil para tests estrictos.
<code>any(), eq(), isNull()</code>	Matchers de argumentos.	<code>verify(repo).find(eq("123"));</code>

Método/Anotación	Función	Ejemplo
ArgumentCaptor	Capturar argumento enviado.	Verificar DTO/evento construido.
InOrder	Verificar orden de llamadas.	Flujo transaccional o protocolo.

4.3 Ejemplo base: servicio con repositorio

Supongamos una arquitectura simple de dominio/aplicación/persistencia. El servicio no debe depender de una base real para ser probado.

```

1 public class Book {
2     private final String isbn;
3     private final String title;
4
5     public Book(String isbn, String title) {
6         if (isbn == null || isbn.isBlank()) {
7             throw new IllegalArgumentException("isbn is required");
8         }
9         if (title == null || title.isBlank()) {
10            throw new IllegalArgumentException("title is required");
11        }
12        this.isbn = isbn;
13        this.title = title;
14    }
15
16    public String getIsbn() { return isbn; }
17    public String getTitle() { return title; }
18 }
19
20 public interface BookRepository {
21     Book findByIsbn(String isbn);
22     void save(Book book);
23 }
24
25 public class BookService {
26     private final BookRepository repository;
27
28     public BookService(BookRepository repository) {
29         this.repository = repository;
30     }
31
32     public Book register(String isbn, String title) {
33         Book existing = repository.findByIsbn(isbn);
34         if (existing != null) {
35             throw new IllegalStateException("book already exists");
36         }
37         Book book = new Book(isbn, title);
38         repository.save(book);

```

```
39     return book;
40 }
41 }
```

4.4 Test con Mockito JUnit4 Runner

```
1  import org.junit.Test;
2  import org.junit.runner.RunWith;
3  import org.mockito.ArgumentCaptor;
4  import org.mockito.InjectMocks;
5  import org.mockito.Mock;
6  import org.mockito.junit.MockitoJUnitRunner;
7
8  import static org.junit.Assert.*;
9  import static org.mockito.Mockito.*;
10
11 @RunWith(MockitoJUnitRunner.class)
12 public class BookServiceTest {
13
14     @Mock
15     private BookRepository repository;
16
17     @InjectMocks
18     private BookService service;
19
20     @Test
21     public void register_savesBook_whenIsbnDoesNotExist() {
22         // Arrange
23         when(repository.findByIsbn("978-1")).thenReturn(null);
24
25         // Act
26         Book created = service.register("978-1", "Effective Java");
27
28         // Assert: estado devuelto
29         assertEquals("978-1", created.getIsbn());
30         assertEquals("Effective Java", created.getTitle());
31
32         // Assert: interaccion con dependencia
33         ArgumentCaptor<Book> captor = ArgumentCaptor.forClass(Book.class);
34         verify(repository).save(captor.capture());
35         assertEquals("978-1", captor.getValue().getIsbn());
36         verify(repository).findByIsbn("978-1");
37         verifyNoMoreInteractions(repository);
38     }
39
40     @Test(expected = IllegalStateException.class)
41     public void register_throwsException_whenBookAlreadyExists() {
42         when(repository.findByIsbn("978-1"))
43             .thenReturn(new Book("978-1", "Existing"));
44
45         service.register("978-1", "New title");
46     }
```

```
47     verify(repository, never()).save(any(Book.class)); // no se alcanza
48 }
49 }
```

Error típico

En el segundo test, la línea después de `service.register(...)` no se ejecuta si se lanza la excepción. Si quieres verificar `never()`, usa `try/catch` o `ExpectedException`; si usas `@Test(expected=...)`, el flujo se corta al lanzar.

4.5 Versión rigurosa para excepción y `never()`

```
1 @Test
2 public void register_doesNotSave_whenBookAlreadyExists() {
3     when(repository.findByIsbn("978-1"))
4         .thenReturn(new Book("978-1", "Existing"));
5
6     try {
7         service.register("978-1", "New title");
8         fail("Expected IllegalStateException");
9     } catch (IllegalStateException ex) {
10         assertEquals("book already exists", ex.getMessage());
11     }
12
13     verify(repository).findByIsbn("978-1");
14     verify(repository, never()).save(any(Book.class));
15     verifyNoMoreInteractions(repository);
16 }
```

4.6 Matchers: regla de consistencia

Cuando usas matchers en una llamada, debes usarlos para todos los argumentos de esa llamada, salvo que uses `eq()` para fijar uno.

```
1 // Correcto
2 verify(repository).findByCategoryAndLimit(eq("java"), anyInt());
3
4 // Incorrecto: mezcla valor crudo y matcher
5 verify(repository).findByCategoryAndLimit("java", anyInt());
```

4.7 Métodos void: `doThrow`, `doNothing`, `doAnswer`

```
1 @Test
2 public void delete_propagatesException_whenRepositoryFails() {
3     doThrow(new IllegalStateException("db unavailable"))
4         .when(repository).deleteByIsbn("978-1");
5
6     try {
```

```
7     service.delete("978-1");
8     fail("Expected exception");
9 } catch (IllegalStateException ex) {
10     assertEquals("db unavailable", ex.getMessage());
11 }
12
13 verify(repository).deleteByIsbn("978-1");
14 }
```

4.8 ArgumentCaptor: cuándo usarlo

Usa ArgumentCaptor cuando la clase bajo prueba construye un objeto internamente y quieres inspeccionar qué fue enviado al colaborador. No lo uses para stubbing si puedes evitarlo; para stubbing, normalmente son más claros los matchers.

```
1 @Test
2 public void publish_buildsCorrectEvent() {
3     EventBus eventBus = mock(EventBus.class);
4     OrderPublisher publisher = new OrderPublisher(eventBus);
5
6     publisher.orderCreated("0-1", 25000);
7
8     ArgumentCaptor<JsonObject> eventCaptor = ArgumentCaptor.forClass(JsonObject.class);
9     verify(eventBus).publish(eq("orders.created"), eventCaptor.capture());
10
11     JsonObject event = eventCaptor.getValue();
12     assertEquals("0-1", event.getString("orderId"));
13     assertEquals(Integer.valueOf(25000), event.getInteger("amount"));
14 }
```

4.9 Spy: uso mínimo y advertencia

Un spy envuelve un objeto real. Si no programas un método, se ejecuta el método real. Por eso puede ser peligroso si el método real accede a red, base de datos o estado externo.

```
1 @Test
2 public void spy_example() {
3     PriceCalculator real = new PriceCalculator();
4     PriceCalculator spy = spy(real);
5
6     // Con spy, preferir doReturn para no ejecutar metodo real durante stubbing
7     doReturn(100).when(spy).basePrice("SKU-1");
8
9     assertEquals(119, spy.priceWithTax("SKU-1"));
10 }
```

En examen

En ejercicios de unit testing, prefiere mock sobre spy. Usa spy solo si tienes una razón clara: quieres conservar parte del comportamiento real y sustituir un método puntual.

5 ArchUnit: DSL arquitectural imprescindible

5.1 Qué resuelve ArchUnit

ArchUnit permite escribir tests que verifican la arquitectura real del código compilado. En vez de revisar manualmente imports y paquetes, se declaran reglas ejecutables. Esto convierte decisiones arquitectónicas en tests que fallan si alguien introduce una dependencia prohibida.

Idea clave

Si JUnit verifica comportamiento y Mockito controla colaboradores, ArchUnit verifica la forma estructural del sistema: quién puede depender de quién.

5.2 Dependencias Maven

Para JUnit4:

```
1 <dependency>
2   <groupId>com.tngtech.archunit</groupId>
3   <artifactId>archunit-junit4</artifactId>
4   <version>1.4.2</version>
5   <scope>test</scope>
6 </dependency>
```

Para JUnit5, si el proyecto lo usa:

```
1 <dependency>
2   <groupId>com.tngtech.archunit</groupId>
3   <artifactId>archunit-junit5</artifactId>
4   <version>1.4.2</version>
5   <scope>test</scope>
6 </dependency>
```

5.3 Plantilla JUnit4 de ArchUnit

```
1 import com.tngtech.archunit.core.importer.ImportOption;
2 import com.tngtech.archunit.junit.AnalyzeClasses;
3 import com.tngtech.archunit.junit.ArchTest;
4 import com.tngtech.archunit.junit.ArchUnitRunner;
5 import com.tngtech.archunit.lang.ArchRule;
6 import org.junit.runner.RunWith;
7
8 import static com.tngtech.archunit.lang.syntax.ArchRuleDefinition.classes;
9 import static com.tngtech.archunit.lang.syntax.ArchRuleDefinition.noClasses;
10
11 @RunWith(ArchUnitRunner.class)
12 @AnalyzeClasses(
13     packages = "cl.ucn.myapp",
```

```

14     importOptions = ImportOption.DoNotIncludeTests.class
15 )
16 public class ArchitectureTest {
17
18     @ArchTest
19     public static final ArchRule services_should_end_with_service =
20         classes().that().resideInAPackage("..service..")
21             .should().haveSimpleNameEndingWith("Service")
22             .because("el nombre debe comunicar el rol arquitectural");
23 }

```

5.4 Por qué usar `ImportOption.DoNotIncludeTests.class`

En examen

`ImportOption.DoNotIncludeTests.class` excluye las clases de test del análisis arquitectural. Esto es importante porque los tests suelen romper intencionalmente restricciones del código productivo: un test puede acceder a clases internas, mocks, fixtures, builders o adaptadores que no deberían formar parte de la arquitectura de producción. La regla arquitectural debe medir el sistema real, no el código de prueba.

5.5 Patrón S-R-C-E-X para escribir reglas

Letra	Elemento	Pregunta que debes responder
S	Scope	¿Qué paquetes se importan con <code>@AnalyzeClasses</code> ?
R	Referente	¿Sobre qué clases aplica <code>classes().that()</code> o <code>noClasses().that()</code> ?
C	Condición	¿Qué deben o no deben hacer con <code>should()</code> ?
E	Evidencia	¿Por qué existe la regla? Usar <code>because(...)</code> .
X	Excepciones	¿Hay deuda técnica congelada o excepciones acotadas?

5.6 Patrones de paquetes

Patrón	Significado
"..domain.."	Cualquier paquete que contenga domain en cualquier nivel.
cl.ucn.bank.."	Todo lo que está bajo el paquete base cl.ucn.bank.
"..adapter.in.web.."	Adaptadores de entrada web, sin importar prefijo/sufijo.
"..adapter.out.persistence"	Adaptadores de salida a persistencia.
"java.."	Clases de Java estándar.
".org.springframework.."	Cualquier clase de Spring. Útil para prohibir frameworks en dominio.

Error típico

Una regla demasiado amplia puede no detectar violaciones. Una regla demasiado estrecha puede fallar por nombres de paquetes irrelevantes. Antes de escribir la regla, inspecciona la estructura real del proyecto.

5.7 DSL base: classes(), noClasses(), that(), should()

```

1 @ArchTest
2 public static final ArchRule domain_should_not_depend_on_infrastructure =
3     noClasses().that().resideInAPackage("..domain..")
4         .should().dependOnClassesThat()
5         .resideInAnyPackage("..repository..", "..controller..", "..adapter..")
6         .because("el dominio debe ser independiente de infraestructura");

```

Lectura estricta: ninguna clase cuyo paquete contenga domain debe depender de clases ubicadas en repository, controller o adapter. Si una entidad importa JpaRepository, @Entity o un DTO web, debe fallar.

5.8 Reglas de naming

```

1 @ArchTest
2 public static final ArchRule controllers_should_end_with_controller =
3     classes().that().resideInAPackage("..controller..")
4         .should().haveSimpleNameEndingWith("Controller")
5         .because("el nombre comunica el rol de entrada HTTP");
6
7 @ArchTest
8 public static final ArchRule repositories_should_end_with_repository =
9     classes().that().resideInAPackage("..repository..")
10        .should().haveSimpleNameEndingWith("Repository")
11        .because("el nombre comunica acceso a persistencia");

```

5.9 Arquitectura por capas clásica

```
1 import static com.tngtech.archunit.library.Architectures.layeredArchitecture;
2
3 @ArchTest
4 public static final ArchRule layered_architecture = layeredArchitecture()
5     .consideringAllDependencies()
6     .layer("Controller").definedBy("..controller..")
7     .layer("Service").definedBy("..service..")
8     .layer("Repository").definedBy("..repository..")
9     .whereLayer("Controller").mayNotBeAccessedByAnyLayer()
10    .whereLayer("Service").mayOnlyBeAccessedByLayers("Controller")
11    .whereLayer("Repository").mayOnlyBeAccessedByLayers("Service")
12    .because("la direccion permitida es Controller -> Service -> Repository");
```

En examen

Si un Controller inyecta un Repository, se salta el Service. En una regla de capas, eso debe fallar porque el repositorio solo puede ser accedido por la capa Service.

5.10 Reglas de acceso directo

A veces no conviene usar todo `layeredArchitecture`; basta con prohibir una dependencia específica.

```
1 @ArchTest
2 public static final ArchRule controllers_should_not_access_repositories =
3     noClasses().that().resideInAPackage("..controller..")
4     .should().dependOnClassesThat()
5     .resideInAPackage("..repository..")
6     .because("los controladores deben delegar casos de uso al servicio");
```

5.11 Dominio puro

```
1 @ArchTest
2 public static final ArchRule domain_should_not_use_frameworks =
3     noClasses().that().resideInAPackage("..domain..")
4     .should().dependOnClassesThat()
5     .resideInAnyPackage(
6         "org.springframework..",
7         "jakarta.persistence..",
8         "javax.persistence..",
9         "org.hibernate.."
10    )
11    .because("el dominio debe probarse sin levantar infraestructura");
```

5.12 Slices: detección de ciclos

```
1 import static com.tngtech.archunit.library.dependencies.SlicesRuleDefinition.slices;
2
3 @ArchTest
4 public static final ArchRule features_should_be_free_of_cycles =
5     slices().matching("cl.ucn.myapp.*")
6         .should().beFreeOfCycles()
7         .because("los ciclos impiden modularidad y evolucion controlada");
```

5.13 Arquitectura hexagonal / onion

```
1 import static com.tngtech.archunit.library.Architectures.onionArchitecture;
2
3 @ArchTest
4 public static final ArchRule hexagonal_architecture = onionArchitecture()
5     .domainModels("..domain.model..")
6     .domainServices("..domain.service..")
7     .applicationServices("..application..")
8     .adapter("web", "..adapter.in.web..")
9     .adapter("persistence", "..adapter.out.persistence..")
10    .adapter("messaging", "..adapter.out.messaging..")
11    .because("las dependencias deben apuntar hacia el nucleo del negocio");
```

5.14 Reglas hexagonales explícitas

```
1 @ArchTest
2 public static final ArchRule web_adapter_should_not_access_persistence_adapter =
3     noClasses().that().resideInAPackage("..adapter.in.web..")
4         .should().dependOnClassesThat()
5             .resideInAPackage("..adapter.out.persistence..")
6             .because("los adaptadores de entrada hablan con casos de uso, no con BD");
7
8 @ArchTest
9 public static final ArchRule persistence_adapter_should_depend_on_output_ports =
10    classes().that().resideInAPackage("..adapter.out.persistence..")
11        .and().haveSimpleNameEndingWith("Repository")
12        .should().dependOnClassesThat()
13            .resideInAPackage("..application.port.out..")
14            .because("la persistencia implementa puertos definidos por la aplicacion");
```

5.15 Condiciones personalizadas

Cuando la DSL estándar no alcanza, se usa ArchCondition. Ejemplo: prohibir JDBC directo en dominio.

```
1 import com.tngtech.archunit.core.domain.JavaClass;
2 import com.tngtech.archunit.lang.ArchCondition;
3 import com.tngtech.archunit.lang.ConditionEvents;
4 import com.tngtech.archunit.lang.SimpleConditionEvent;
```

```
5
6 static final ArchCondition<JavaClass> not_use_jdbc =
7     new ArchCondition<JavaClass>("not use JDBC directly") {
8         @Override
9         public void check(JavaClass item, ConditionEvents events) {
10             boolean usesJdbc = item.getDirectDependenciesFromSelf().stream()
11                 .anyMatch(dep -> dep.getTargetClass()
12                     .getPackageName().startsWith("java.sql"));
13
14             if (usesJdbc) {
15                 events.add(SimpleConditionEvent.violated(
16                     item, item.getName() + " uses java.sql directly"));
17             }
18         }
19     };
20
21 @ArchTest
22 public static final ArchRule domain_should_not_use_jdbc =
23     classes().that().resideInAPackage("..domain..")
24         .should(not_use_jdbc)
25         .because("el dominio no debe conocer detalles de acceso a datos");
```

5.16 Freezing rules

```
1 import com.tngtech.archunit.library.freeze.FreezingArchRule;
2
3 @ArchTest
4 public static final ArchRule frozen_domain_rule =
5     FreezingArchRule.freeze(domain_should_not_use_frameworks);
```

Congelar no significa declarar correcto el diseño. Significa aceptar violaciones históricas para que no aparezcan violaciones nuevas sin control. En un proyecto formativo limpio, normalmente no deberías congelar reglas salvo que el enunciado lo indique.

5.17 Checklist ArchUnit

- ¿El packages de @AnalyzeClasses coincide con el paquete base real?
- ¿Excluí tests con ImportOption.DoNotIncludeTests.class?
- ¿La regla usa because(...) con justificación arquitectural?
- ¿Estoy verificando dependencias reales, no solo nombres?
- ¿La regla detecta la violación típica del enunciado?
- ¿La regla es lo bastante estricta sin bloquear dependencias legítimas como java..?

6 Vert.x: programación reactiva y EventBus

6.1 Modelo mental

Vert.x permite implementar aplicaciones reactivas sobre la JVM. El modelo central consiste en **verticles** que procesan eventos en un **event loop**; se comunican mediante un **EventBus**. El diseño esperado es no bloqueante: no debes dormir hilos ni esperar activamente respuestas.

Idea clave

En Vert.x la pregunta de examen suele ser: ¿qué patrón de comunicación corresponde? Si es un único consumidor, usa `send`. Si son muchos consumidores, usa `publish`. Si necesitas retorno, usa `request` y `reply`.

6.2 Métodos esenciales de Vert.x

Método/Clase	Función	Patrón
<code>Vertx.vertx()</code>	Crea instancia del núcleo Vert.x.	Arranque
<code>AbstractVerticle</code>	Clase base para componentes reactivos.	Verticle
<code>start(Promise<Void>)</code>	Inicializa el verticle.	Ciclo de vida
<code>startPromise.complete()</code>	Declara inicio exitoso.	Ciclo de vida
<code>startPromise.fail(cause)</code>	Declara fallo de inicio.	Ciclo de vida
<code>deployVerticle(...)</code>	Despliega verticle.	Arranque
<code>DeploymentOptions().setInstances(...)</code>	Despliega múltiples instancias.	Escalamiento
<code>vertx.eventBus()</code>	Obtiene EventBus.	Mensajería
<code>eventBus.consumer(address, handler)</code>	Registra receptor.	Todos
<code>eventBus.send(address, msg)</code>	Envía a un consumidor.	Point-to-Point
<code>eventBus.publish(address, msg)</code>	Difunde a todos los consumidores.	Pub/Sub
<code>eventBus.request(address, msg, handler)</code>	Envía y espera respuesta no bloqueante.	Request-Response
<code>message.reply(response)</code>	Responde a una petición.	Request-Response

Método/Clase	Función	Patrón
vertx.setPeriodic(ms, handler)	Ejecuta tarea periódica.	Eventos temporales
vertx.setTimer(ms, handler)	Ejecuta tarea una vez.	Eventos temporales
JsonObject	Mensaje estructurado reco- mendado.	Payload
Future, Promise	Composición asíncrona.	Control de flujo

6.3 Point-to-Point con send

Semántica: un mensaje se envía a una dirección y solo un consumidor lo recibe. Si hay varios consumidores registrados en la misma dirección, Vert.x escoge uno con estrategia de balanceo tipo round-robin no estricta.

```

1 import io.vertx.core.AbstractVerticle;
2 import io.vertx.core.Promise;
3 import io.vertx.core.Vertx;
4 import org.slf4j.Logger;
5 import org.slf4j.LoggerFactory;
6
7 public class PointToPointApp {
8
9     static final String ADDRESS = "tasks.process";
10
11     public static void main(String[] args) {
12         Vertx vertx = Vertx.vertx();
13         vertx.deployVerticle(new SenderVerticle());
14         vertx.deployVerticle(new WorkerVerticle("worker-1"));
15         vertx.deployVerticle(new WorkerVerticle("worker-2"));
16     }
17
18     public static class SenderVerticle extends AbstractVerticle {
19         @Override
20         public void start(Promise<Void> startPromise) {
21             startPromise.complete();
22
23             vertx.setPeriodic(1000, id -> {
24                 vertx.eventBus().send(ADDRESS, "process one task");
25             });
26         }
27     }
28
29     public static class WorkerVerticle extends AbstractVerticle {
30         private static final Logger LOG = LoggerFactory.getLogger(WorkerVerticle.class);
31         private final String name;
32
33         public WorkerVerticle(String name) {

```

```

34     this.name = name;
35 }
36
37 @Override
38 public void start(Promise<Void> startPromise) {
39     vertx.eventBus().consumer(ADDRESS, message -> {
40         LOG.info("{} received: {}", name, message.body());
41     });
42     startPromise.complete();
43 }
44 }
45 }

```

En examen

Usa send cuando el mensaje representa una tarea que debe procesar un solo receptor: validación, cálculo, envío de correo, procesamiento de orden, carga balanceada entre workers.

6.4 Publish/Subscribe con publish

Semántica: un evento se publica en una dirección y todos los consumidores registrados reciben una copia. No se espera respuesta.

```

1  import io.vertx.core.AbstractVerticle;
2  import io.vertx.core.DeploymentOptions;
3  import io.vertx.core.Promise;
4  import io.vertx.core.Vertx;
5  import io.vertx.core.json.JsonObject;
6  import org.slf4j.Logger;
7  import org.slf4j.LoggerFactory;
8
9  public class PublishSubscribeApp {
10
11     static final String ADDRESS = "orders.created";
12
13     public static void main(String[] args) {
14         Vertx vertx = Vertx.vertx();
15         vertx.deployVerticle(new OrderPublisher());
16         vertx.deployVerticle(new EmailSubscriber());
17         vertx.deployVerticle(AuditSubscriber.class.getName(),
18             new DeploymentOptions().setInstances(2));
19     }
20
21     public static class OrderPublisher extends AbstractVerticle {
22         @Override
23         public void start(Promise<Void> startPromise) {
24             startPromise.complete();
25             vertx.setPeriodic(5000, id -> {
26                 JsonObject event = new JsonObject()
27                     .put("orderId", "0-" + id)
28                     .put("amount", 25000);

```

```

29         vertx.eventBus().publish(ADDRESS, event);
30     });
31 }
32 }
33
34 public static class EmailSubscriber extends AbstractVerticle {
35     private static final Logger LOG = LoggerFactory.getLogger(EmailSubscriber.class);
36
37     @Override
38     public void start(Promise<Void> startPromise) {
39         vertx.eventBus().<JsonObject>consumer(ADDRESS, message -> {
40             LOG.info("Send email for event: {}", message.body().encode());
41         });
42         startPromise.complete();
43     }
44 }
45
46 public static class AuditSubscriber extends AbstractVerticle {
47     private static final Logger LOG = LoggerFactory.getLogger(AuditSubscriber.class);
48
49     @Override
50     public void start(Promise<Void> startPromise) {
51         vertx.eventBus().<JsonObject>consumer(ADDRESS, message -> {
52             LOG.info("Audit event: {}", message.body().encode());
53         });
54         startPromise.complete();
55     }
56 }
57 }

```

En examen

Usa publish cuando el mensaje es un evento de dominio o notificación: order.created, payment.accepted, ticket.sold. Varios subsistemas pueden reaccionar sin que el publicador los conozca.

6.5 Request-Response con request y reply

Semántica: un verticle envía una solicitud y registra un callback para procesar la respuesta. No debe bloquear el hilo esperando.

```

1 import io.vertx.core.AbstractVerticle;
2 import io.vertx.core.Promise;
3 import io.vertx.core.Vertx;
4 import io.vertx.core.eventbus.EventBus;
5 import io.vertx.core.json.JsonObject;
6 import org.slf4j.Logger;
7 import org.slf4j.LoggerFactory;
8
9 public class RequestResponseApp {
10
11     static final String ADDRESS = "books.validate";

```

```
12
13 public static void main(String[] args) {
14     Vertx vertx = Vertx.vertx();
15     vertx.deployVerticle(new ValidationServiceVerticle());
16     vertx.deployVerticle(new ClientVerticle());
17 }
18
19 public static class ClientVerticle extends AbstractVerticle {
20     private static final Logger LOG = LoggerFactory.getLogger(ClientVerticle.class);
21
22     @Override
23     public void start(Promise<Void> startPromise) {
24         EventBus eventBus = vertx.eventBus();
25         JsonObject request = new JsonObject()
26             .put("isbn", "978-1")
27             .put("title", "Effective Java");
28
29         eventBus.<JsonObject>request(ADDRESS, request, reply -> {
30             if (reply.succeeded()) {
31                 LOG.info("Validation response: {}", reply.result().body().encode());
32             } else {
33                 LOG.error("Validation failed", reply.cause());
34             }
35         });
36
37         startPromise.complete();
38     }
39 }
40
41 public static class ValidationServiceVerticle extends AbstractVerticle {
42     @Override
43     public void start(Promise<Void> startPromise) {
44         vertx.eventBus().<JsonObject>consumer(ADDRESS, message -> {
45             JsonObject book = message.body();
46             boolean valid = book.getString("isbn") != null
47                 && book.getString("title") != null;
48
49             message.reply(new JsonObject().put("valid", valid));
50         });
51         startPromise.complete();
52     }
53 }
54 }
```

En examen

Usa request/reply cuando necesitas un resultado: validar datos, consultar stock, calcular precio, autenticar credenciales. No lo uses para notificaciones globales: eso es publish.

6.6 Future y Promise: composición asíncrona

```
1 public Future<Book> findBook(String isbn) {
2     Promise<Book> promise = Promise.promise();
3
4     vertx.eventBus().<JsonObject>request("books.find", isbn, reply -> {
5         if (reply.succeeded()) {
6             JsonObject json = reply.result().body();
7             promise.complete(new Book(json.getString("isbn"), json.getString("title")));
8         } else {
9             promise.fail(reply.cause());
10        }
11    });
12
13    return promise.future();
14 }
```

6.7 Reglas de diseño Vert.x

- No bloquear el event loop con `Thread.sleep`, I/O síncrono pesado o loops largos.
- Usar `JsonObject` para mensajes estructurados, especialmente si habrá interoperabilidad.
- Declarar direcciones como constantes: evita errores por strings duplicados.
- Manejar `succeeded()` y `failed()` en operaciones asíncronas.
- En `request`, responder siempre con `reply` o fallar de forma explícita si el protocolo lo requiere.
- Separar verticles por responsabilidad: publicador, consumidor, servicio, adaptador.

6.8 Test de Vert.x con JUnit4 y vertx-unit

Si el proyecto usa Vert.x Unit con JUnit4, la estructura típica es:

```
1 import io.vertx.core.Vertx;
2 import io.vertx.ext.unit.Async;
3 import io.vertx.ext.unit.TestContext;
4 import io.vertx.ext.unit.junit.VertxUnitRunner;
5 import org.junit.After;
6 import org.junit.Before;
7 import org.junit.Test;
8 import org.junit.runner.RunWith;
9
10 @RunWith(VertxUnitRunner.class)
11 public class RequestResponseVerticleTest {
12
13     private Vertx vertx;
14
15     @Before
16     public void setUp(TestContext context) {
17         vertx = Vertx.vertx();
18     }
19 }
```

```
18     vertx.deployVerticle(new RequestResponseApp.ValidationServiceVerticle(),
19         context.asyncAssertSuccess());
20 }
21
22 @After
23 public void tearDown(TestContext context) {
24     vertx.close(context.asyncAssertSuccess());
25 }
26
27 @Test
28 public void validation_returnsTrue_forValidBook(TestContext context) {
29     Async async = context.async();
30
31     JsonObject request = new JsonObject()
32         .put("isbn", "978-1")
33         .put("title", "Effective Java");
34
35     vertx.eventBus().<JsonObject>request("books.validate", request, reply -> {
36         context.assertTrue(reply.succeeded());
37         context.assertTrue(reply.result().body().getBoolean("valid"));
38         async.complete();
39     });
40 }
41 }
```

Error típico

En tests asíncronos, no termines el test antes de que llegue el callback. Debes usar `Async async = context.async()` y llamar `async.complete()` dentro del handler.

7 Integración: ejemplo compacto de arquitectura + JUnit + Mockito + ArchUnit + Vert.x

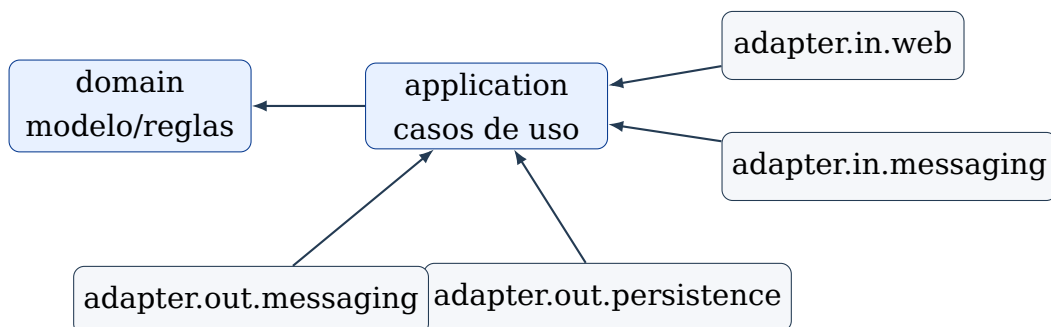
7.1 Estructura de paquetes recomendada

```

1 cl.ucn.books
2   domain
3     model
4       Book.java
5   service
6     IsbnPolicy.java
7   application
8     BookUseCase.java
9     BookApplicationService.java
10  port
11    out
12      BookRepositoryPort.java
13      EventPublisherPort.java
14  adapter
15    in
16      web
17        BookController.java
18      messaging
19        BookCommandConsumer.java
20    out
21      persistence
22        JpaBookRepositoryAdapter.java
23      messaging
24        VertxEventPublisherAdapter.java

```

7.2 Regla de dependencia esperada



7.3 Servicio de aplicación

```

1 public interface BookRepositoryPort {
2     Book findByIsbn(String isbn);
3     void save(Book book);
4 }
5

```

```
6 public interface EventPublisherPort {
7     void publishBookRegistered(Book book);
8 }
9
10 public class BookApplicationService {
11     private final BookRepositoryPort repository;
12     private final EventPublisherPort publisher;
13
14     public BookApplicationService(BookRepositoryPort repository,
15                                   EventPublisherPort publisher) {
16         this.repository = repository;
17         this.publisher = publisher;
18     }
19
20     public Book register(String isbn, String title) {
21         if (repository.findByIsbn(isbn) != null) {
22             throw new IllegalStateException("book already exists");
23         }
24
25         Book book = new Book(isbn, title);
26         repository.save(book);
27         publisher.publishBookRegistered(book);
28         return book;
29     }
30 }
```

7.4 Test unitario con Mockito

```
1 @RunWith(MockitoJUnitRunner.class)
2 public class BookApplicationServiceTest {
3
4     @Mock private BookRepositoryPort repository;
5     @Mock private EventPublisherPort publisher;
6
7     @InjectMocks private BookApplicationService service;
8
9     @Test
10     public void register_savesAndPublishesEvent_whenBookIsNew() {
11         when(repository.findByIsbn("978-1")).thenReturn(null);
12
13         Book result = service.register("978-1", "Effective Java");
14
15         assertEquals("978-1", result.getIsbn());
16         verify(repository).save(any(Book.class));
17         verify(publisher).publishBookRegistered(any(Book.class));
18         verifyNoMoreInteractions(publisher);
19     }
20
21     @Test
22     public void register_doesNotSaveNorPublish_whenBookExists() {
23         when(repository.findByIsbn("978-1"))
24             .thenReturn(new Book("978-1", "Existing"));
```

```
25
26     try {
27         service.register("978-1", "Effective Java");
28         fail("Expected IllegalStateException");
29     } catch (IllegalStateException expected) {
30         assertEquals("book already exists", expected.getMessage());
31     }
32
33     verify(repository).findByIsbn("978-1");
34     verify(repository, never()).save(any(Book.class));
35     verifyNoInteractions(publisher);
36 }
37 }
```

7.5 Adaptador Vert.x de salida

```
1 public class VertxEventPublisherAdapter implements EventPublisherPort {
2     private static final String ADDRESS = "books.registered";
3     private final EventBus eventBus;
4
5     public VertxEventPublisherAdapter(EventBus eventBus) {
6         this.eventBus = eventBus;
7     }
8
9     @Override
10    public void publishBookRegistered(Book book) {
11        JsonObject event = new JsonObject()
12            .put("isbn", book.getIsbn())
13            .put("title", book.getTitle());
14
15        eventBus.publish(ADDRESS, event);
16    }
17 }
```

7.6 Test del adaptador con Mockito

```
1 public class VertxEventPublisherAdapterTest {
2
3     @Test
4     public void publishBookRegistered_publishesJsonEvent() {
5         EventBus eventBus = mock(EventBus.class);
6         VertxEventPublisherAdapter adapter = new VertxEventPublisherAdapter(eventBus);
7
8         adapter.publishBookRegistered(new Book("978-1", "Effective Java"));
9
10        ArgumentCaptor<JsonObject> captor = ArgumentCaptor.forClass(JsonObject.class);
11        verify(eventBus).publish(eq("books.registered"), captor.capture());
12        assertEquals("978-1", captor.getValue().getString("isbn"));
13        assertEquals("Effective Java", captor.getValue().getString("title"));
14    }
15 }
```

```
15 }
```

7.7 Test arquitectural

```
1 @RunWith(ArchUnitRunner.class)
2 @AnalyzeClasses(
3     packages = "cl.ucn.books",
4     importOptions = ImportOption.DoNotIncludeTests.class
5 )
6 public class BookArchitectureTest {
7
8     @ArchTest
9     public static final ArchRule domain_should_not_depend_on_application_or_adapters =
10         noClasses().that().resideInAPackage("..domain..")
11             .should().dependOnClassesThat()
12             .resideInAnyPackage("..application..", "..adapter..")
13             .because("el dominio es el nucleo estable del sistema");
14
15     @ArchTest
16     public static final ArchRule web_should_not_access_persistence =
17         noClasses().that().resideInAPackage("..adapter.in.web..")
18             .should().dependOnClassesThat()
19             .resideInAPackage("..adapter.out.persistence..")
20             .because("un adaptador de entrada no debe saltarse los casos de uso");
21
22     @ArchTest
23     public static final ArchRule adapters_should_not_depend_on_each_other =
24         noClasses().that().resideInAPackage("..adapter..")
25             .should().dependOnClassesThat()
26             .resideInAnyPackage("..adapter.in..", "..adapter.out..")
27             .because("los adaptadores se coordinan mediante la aplicacion, no entre si");
28 }
```

Error típico

La última regla puede ser demasiado amplia si un proyecto separa adaptadores de forma distinta. En tal caso, conviene escribir reglas específicas: web no accede persistence, messaging in no accede persistence, etc. La regla debe reflejar la arquitectura real pedida por el enunciado.

8 Tablas tipo formulario

8.1 JUnit4 en una página

Elemento	Uso rápido
@Test	Declara test. Puede usar expected y timeout, aunque se recomienda cuidado con timeout.
@Before	Fixture antes de cada test.
@After	Limpieza después de cada test.
assertEquals(e,a)	Compara esperado y actual.
assertTrue/False	Verifica condición booleana.
assertNull/NotNull	Verifica nulidad.
assertSame/NotSame	Verifica identidad de referencia.
assertArrayEquals	Compara arreglos.
fail	Falla explícitamente; útil en try/catch de excepciones.
@RunWith	Runner externo: Mockito, Parameterized, VertxUnit, ArchUnit.
@Rule ExpectedException	Excepción con tipo y mensaje.

8.2 Mockito en una página

Elemento	Uso rápido
mock() / @Mock	Crear dependencia simulada.
@InjectMocks	Instanciar clase bajo prueba con mocks.
when(x).thenReturn(y)	Programar retorno.
when(x).thenThrow(e)	Programar excepción en método no void.
doThrow(e).when(m).void()	Programar excepción en método void.
verify(mock).m()	Verificar que ocurrió llamada.
times(n)	Cantidad exacta.
never()	Verificar que no ocurrió.
any(), eq(), argThat()	Matchers de argumentos.
ArgumentCaptor	Capturar objeto construido y enviado.
InOrder	Verificar protocolo de llamadas.

Elemento	Uso rápido
<code>verifyNoMoreInteractions</code>	Cerrar test estrictamente.

8.3 ArchUnit en una página

Elemento	Uso rápido
<code>@AnalyzeClasses(packages=...)</code>	Define qué clases analizar.
<code>ImportOption.DoNotIncludeTests</code>	Excluye tests del análisis.
<code>@ArchTest</code>	Marca regla arquitectural.
<code>classes().that()</code>	Selecciona clases que deben cumplir algo.
<code>noClasses().that()</code>	Selecciona clases que no deben hacer algo.
<code>resideInAPackage</code>	Filtra por paquete.
<code>dependOnClassesThat()</code>	Verifica dependencias.
<code>onlyDependOnClassesThat()</code>	Lista cerrada de dependencias permitidas.
<code>onlyBeAccessed().byAnyPackage()</code>	Controla quién puede acceder a un paquete.
<code>haveSimpleNameEndingWith()</code>	Regla de naming.
<code>layeredArchitecture()</code>	Reglas por capas.
<code>onionArchitecture()</code>	Hexagonal/onion.
<code>slices().matching()</code>	Detectar ciclos por módulo/feature.
<code>because()</code>	Justificación arquitectural.

8.4 Vert.x EventBus en una página

Elemento	Uso rápido
<code>Vertx.vertx()</code>	Crear instancia Vert.x.
<code>deployVerticle</code>	Desplegar componente reactivo.
<code>AbstractVerticle</code>	Clase base de verticle.
<code>start(Promise<Void>)</code>	Inicialización asíncrona.
<code>eventBus.consumer</code>	Registrar receptor en dirección.
<code>eventBus.send</code>	Point-to-Point: un receptor.
<code>eventBus.publish</code>	Publish/Subscribe: todos reciben copia.
<code>eventBus.request</code>	Solicitud con respuesta no bloqueante.
<code>message.reply</code>	Responder una petición.

Elemento	Uso rápido
<code>setPeriodic</code>	Ejecutar acción periódica.
<code>DeploymentOptions.setInstances</code>	Verificar las instancias de un verticle.
<code>JsonObject</code>	Payload estructurado.

9 Errores frecuentes y cómo corregirlos

9.1 Errores JUnit4

Error	Corrección
Test sin assert.	Agregar <code>assert*</code> , <code>verify</code> o validación automática.
Test depende de la consola.	Reemplazar <code>println</code> por aserciones.
Fixture compartido mutable.	Inicializar en <code>@Before</code> .
Demasiados asserts inconexos.	Separar casos o justificar que son propiedades del mismo resultado.
<code>@Test(expected)</code> con muchas líneas.	Usar <code>ExpectedException</code> o <code>try/catch</code> con <code>fail</code> .
Test lento por red/BD.	Convertir en unitario con Mockito o moverlo a integración.

9.2 Errores Mockito

Error	Corrección
Mezclar matchers y valores crudos.	Usar <code>eq(valor)</code> junto con <code>any()</code> .
Mockear la clase bajo prueba.	Mockear colaboradores, no la unidad que quieres probar.
Verificar después de excepción con <code>@Test(expected)</code> .	Usar <code>try/catch</code> para poder verificar después.
Usar <code>spy</code> sin necesidad.	Preferir <code>mock</code> ; si usas <code>spy</code> , <code>stubbing</code> con <code>doReturn</code> .
No verificar interacción importante.	Agregar <code>verify</code> sobre repositorio, publicador o gateway.
Abusar de <code>verifyNoMoreInteractions</code> .	Usarlo solo si la ausencia de otras llamadas importa.

9.3 Errores ArchUnit

Error	Corrección
Paquete base incorrecto.	Revisar package real de las clases.
No excluir tests.	Usar <code>ImportOption.DoNotIncludeTests.class</code> .
Regla sin because.	Agregar justificación arquitectural clara.
Usar <code>resideInAPackage("service")</code> sin puntos.	Usar patrones como <code>"..service.."</code> .
Regla demasiado amplia.	Dividir en reglas específicas.
Confundir acceso con dependencia.	Leer mensaje de violación: import, campo, llamada, anotación, herencia, etc.

9.4 Errores Vert.x

Error	Corrección
Usar <code>publish</code> esperando respuesta.	Cambiar a <code>request/reply</code> .
Usar <code>send</code> para notificar a varios.	Cambiar a <code>publish</code> .
Bloquear el event loop con <code>Thread.sleep</code> .	Usar <code>setTimer</code> , <code>setPeriodic</code> o APIs asíncronas.
No manejar fallo de request.	Revisar <code>reply.succeeded()</code> y <code>reply.failed()</code> .
Direcciones hardcodeadas en muchas clases.	Definir constantes compartidas.
Test asíncrono termina antes del callback.	Usar <code>Async</code> y <code>async.complete()</code> .

10 Estrategia de resolución para pruebas del curso

10.1 Lectura del enunciado

Antes de escribir código, marca con lápiz o mentalmente:

1. **Arquitectura esperada:** capas, hexagonal, microservicios, mensajería.
2. **Dependencias prohibidas:** controlador-repositorio, dominio-framework, adaptador-adaptador.
3. **Unidad bajo prueba:** clase de dominio, servicio, adaptador, verticle.
4. **Colaboradores:** repositorios, gateways, EventBus, clientes externos.
5. **Tipo de test:** unitario, integración, arquitectura, reactivo.
6. **Patrón Vert.x:** point-to-point, publish-subscribe o request-response.

10.2 Orden recomendado de implementación

Receta

1. Crear modelo mínimo de dominio con validaciones simples.
2. Definir interfaces/puertos antes de adaptadores concretos.
3. Implementar servicio/caso de uso.
4. Escribir tests JUnit4 del dominio.
5. Escribir tests Mockito del servicio.
6. Implementar adaptador Vert.x si corresponde.
7. Escribir regla ArchUnit para proteger la estructura.
8. Ejecutar `mvn test` y corregir violaciones.

10.3 Cómo saber si un test es unitario

Un test unitario real:

- prueba una clase o unidad pequeña;
- no depende de red, base de datos ni servidor real;
- usa mocks/fakes si hay dependencias externas;
- es rápido y determinístico;
- falla por una razón clara.

Si levantas una base, un servidor HTTP o múltiples componentes reales, probablemente estás en test de integración o sistema.

10.4 Cómo saber si una regla ArchUnit está bien planteada

Una regla está bien planteada si:

- expresa una decisión arquitectural real del enunciado;
- detecta una violación que el profesor esperaría corregir;
- no depende de detalles accidentales;
- tiene un because que enseña la razón;
- falla con un mensaje útil.

10.5 Cómo decidir entre send, publish y request

Pregunta	Respuesta	Método
¿Debe procesarlo solo un receptor?	Sí, es una tarea o comando.	send
¿Deben enterarse muchos receptores?	Sí, es un evento/notificación.	publish
¿Necesito un resultado de vuelta?	Sí, es consulta/validación/RPC interna.	request + reply
¿El emisor conoce al receptor concreto?	No debería; solo conoce dirección lógica.	EventBus
¿El receptor puede no estar activo?	En EventBus local no hay persistencia por defecto.	Considerar broker real si se requiere durabilidad.

11 Apéndice A: dependencias Maven de referencia

```
1 <dependencies>
2   <!-- JUnit 4 -->
3   <dependency>
4     <groupId>junit</groupId>
5     <artifactId>junit</artifactId>
6     <version>4.13.2</version>
7     <scope>test</scope>
8   </dependency>
9
10  <!-- Mockito para JUnit 4 -->
11  <dependency>
12    <groupId>org.mockito</groupId>
13    <artifactId>mockito-core</artifactId>
14    <version>5.18.0</version>
15    <scope>test</scope>
16  </dependency>
17
18  <!-- ArchUnit con JUnit 4 -->
19  <dependency>
20    <groupId>com.tngtech.archunit</groupId>
21    <artifactId>archunit-junit4</artifactId>
22    <version>1.4.2</version>
23    <scope>test</scope>
24  </dependency>
25
26  <!-- Vert.x Core -->
27  <dependency>
28    <groupId>io.vertx</groupId>
29    <artifactId>vertx-core</artifactId>
30    <version>5.0.2</version>
31  </dependency>
32
33  <!-- Vert.x Unit para JUnit 4, si el proyecto lo requiere -->
34  <dependency>
35    <groupId>io.vertx</groupId>
36    <artifactId>vertx-unit</artifactId>
37    <version>4.5.15</version>
38    <scope>test</scope>
39  </dependency>
40 </dependencies>
```

Error típico

Las versiones deben ajustarse al pom.xml del proyecto entregado. En una evaluación, no cambies versiones salvo que el enunciado lo permita. Lo importante para el examen es reconocer dependencias, scopes y APIs.

12 Apéndice B: plantilla completa de test arquitectural

```
1 @RunWith(ArchUnitRunner.class)
2 @AnalyzeClasses(
3     packages = "cl.ucn.project",
4     importOptions = ImportOption.DoNotIncludeTests.class
5 )
6 public class ArchitectureTest {
7
8     @ArchTest
9     public static final ArchRule controllers_should_only_access_services =
10         classes().that().resideInAPackage("..controller..")
11             .should().onlyDependOnClassesThat()
12                 .resideInAnyPackage(
13                     "..controller..",
14                     "..service..",
15                     "..dto..",
16                     "java..",
17                     "javax..",
18                     "jakarta..",
19                     "org.springframework.."
20                 )
21             .because("los controladores solo orquestan entrada y delegan al servicio");
22
23     @ArchTest
24     public static final ArchRule services_should_not_access_controllers =
25         noClasses().that().resideInAPackage("..service..")
26             .should().dependOnClassesThat()
27                 .resideInAPackage("..controller..")
28             .because("la capa de aplicacion no debe depender de entrada web");
29
30     @ArchTest
31     public static final ArchRule repositories_should_only_be_accessed_by_services =
32         classes().that().resideInAPackage("..repository..")
33             .should().onlyBeAccessed().byAnyPackage("..service..", "..repository..")
34             .because("la persistencia debe quedar detras de la capa de servicio");
35 }
```

13 Apéndice C: plantilla completa Mockito

```
1 @RunWith(MockitoJUnitRunner.class)
2 public class ServiceTest {
3
4     @Mock private Repository repository;
5     @Mock private EventPublisher publisher;
6     @InjectMocks private Service service;
7
8     @Test
9     public void method_returnsExpectedValue_whenCondition() {
10         when(repository.find("id-1")).thenReturn(new Entity("id-1"));
11     }
```

```
12     Result result = service.method("id-1");
13
14     assertEquals("id-1", result.getId());
15     verify(repository).find("id-1");
16     verify(publisher).publish(any(Event.class));
17 }
18
19 @Test
20 public void method_throwsException_whenDependencyFails() {
21     when(repository.find("id-1"))
22         .thenReturn(new IllegalStateException("repository failed"));
23
24     try {
25         service.method("id-1");
26         fail("Expected IllegalStateException");
27     } catch (IllegalStateException ex) {
28         assertEquals("repository failed", ex.getMessage());
29     }
30
31     verify(repository).find("id-1");
32     verifyNoInteractions(publisher);
33 }
34 }
```

14 Conclusión

El núcleo de la materia no está en memorizar llamadas sueltas, sino en entender qué problema resuelve cada DSL:

- **JUnit4** expresa expectativas de comportamiento.
- **Mockito** permite aislar la unidad y observar sus colaboraciones.
- **ArchUnit** convierte decisiones de arquitectura en reglas ejecutables.
- **Vert.x** permite modelar comunicación reactiva: comandos, eventos y solicitudes con respuesta.

La forma más segura de resolver ejercicios es partir por la arquitectura esperada, identificar dependencias prohibidas, diseñar puertos o interfaces cuando haya inversión de dependencias, y después escribir tests que prueben comportamiento y estructura. En términos prácticos: si el código compila pero viola la arquitectura, está incompleto; si la arquitectura está declarada pero no hay tests de comportamiento, también está incompleto; si hay comunicación reactiva pero se eligió mal send, publish o request, el diseño no representa el patrón pedido.

Idea clave

La consigna final para estudiar es: **cada herramienta protege una dimensión distinta del software**. JUnit protege comportamiento; Mockito protege aislamiento; ArchUnit protege arquitectura; Vert.x materializa comunicación reactiva.

Referencias

1. Daniel San Martín, *Arquitectura de Software*, material de clase de Patrones de Software y Programación.
2. Daniel San Martín, *Testing*, material de clase de Patrones de Software y Programación.
3. Daniel San Martín, *Introducción a la Programación Reactiva*, material de clase de Patrones de Software y Programación.
4. Daniel San Martín, *ArchUnit - Chequeo arquitectural automatizado en Java y validación de arquitectura hexagonal*, material de clase de Patrones de Software y Programación.
5. JUnit 4, *JUnit 4.13.2 API* y documentación de `org.junit.Test/org.junit.Assert`. Disponible en <https://junit.org/junit4/> y <https://junit.org/javadoc/latest/org/junit/Assert.html>.
6. Mockito, *Mockito Javadoc*. Disponible en <https://site.mockito.org/javadoc/current/org/mockito/Mockito.html>.
7. Mockito, *ArgumentCaptor Javadoc*. Disponible en <https://site.mockito.org/javadoc/current/org/mockito/ArgumentCaptor.html>.
8. Eclipse Vert.x, *Vert.x Core Manual*. Disponible en <https://vertx.io/docs/vertx-core/java/>.
9. Eclipse Vert.x, *EventBus API*. Disponible en <https://vertx.io/docs/apidocs/io/vertx/core/eventbus/EventBus.html>.
10. ArchUnit, *ArchUnit User Guide*. Disponible en https://www.archunit.org/userguide/html/000_Index.html.